

SOCIAL NETWORKS

NPTEL Online Course • Prof. Sudarshan Iyengar, IIT Ropar

Week 2 — Question Paper (Comprehension-Based MCQ / MSQ)

Comprehension 1 — Reading and Measuring Real-World Networks

Case Study

Tomás works in a lab that studies how real systems hold together. From public data portals he pulls four networks, each saved in a different file convention: an air-traffic network whose nodes are airports and whose links are non-stop routes (a plain two-column text file); a food web whose nodes are species and whose links record who preys on whom (GraphML); a national power grid whose nodes are substations joined by transmission lines (Pajek); and a research co-authorship network whose nodes are scientists, joined whenever they have written a paper together (GML).

He is careful about direction. Some links are inherently mutual — two airports joined by a route, two substations joined by a line, two scientists who co-authored a paper — and are modelled as undirected. Others are inherently one-way — a predator feeding on its prey — and must be modelled as directed. He reads each file with the NetworkX reader that fits its format, skims the headline counts, then asks the same structural questions of every graph: how lopsided the per-node link counts are, how full the graph is against the most links it could carry, how tightly a node's neighbours connect among themselves, and how many hops separate the two most distant nodes.

To keep his hand calculations honest, Tomás sketches one tiny relay network on paper — eight stations labelled A through H, joined by exactly these nine links:

A-B, A-C, A-D, B-C, D-E, E-F, F-G, G-H, E-G

He expects each real network's link-count profile to be lopsided in the familiar way — a few heavily connected hubs and a long tail of lightly connected nodes.

Questions 1–10

Q1. [MCQ] Among Tomás's four networks, which one must be modelled as a directed graph, and why?

- A. The air-traffic network, because each route serves two airports.
- B. The power-grid network, because a transmission line connects two substations.
- C. The food web, because “species X preys on species Y” does not imply “Y preys on X” — the relation runs one way.
- D. The co-authorship network, because two scientists either have or have not written a paper together.

Answer: C

Reference: Lecture 19 – Datasets: Different Formats (≈01:31, directed vs undirected)

Solution: A relation needs a directed edge when it is one-way. Predation is one-way: X preying on Y says nothing about Y preying on X, so the food web is directed (C). The other three describe mutual, symmetric relations — a shared route, a shared line, a shared paper — which are modelled with undirected edges.

Q2. [MCQ] The air-traffic file stores one route per line as a pair of airport codes (for example, 'DEL BOM'). Which NetworkX function reads it into a graph object?

- A. `nx.read_edgelist(filename)`
- B. `nx.read_adjlist(filename)`
- C. `nx.read_gml(filename)`
- D. `nx.read_graphml(filename)`

Answer: A

Reference: Lecture 21 – Datasets: Analyzing Using NetworkX (≈02:00)

Solution: A file with one edge per row, given as a (source, target) pair, is in edge-list format, read with `nx.read_edgelist`. `read_adjlist` expects a node followed by its neighbours on each row; `read_gml` and `read_graphml` handle the GML and GraphML formats.

Q3. [MSQ] Tomás reviews how the formats differ. Which statements are correct? (Select all that apply.)

- I. In an edge-list file, each row lists two node IDs representing a single edge (source, target).
 - II. In an adjacency-list file, the first ID in a row is a node and the remaining IDs on that row are the nodes adjacent to it.
 - III. A plain edge-list or adjacency-list cannot attach non-numeric labels or colour attributes to nodes; richer formats such as GML or GraphML can.
 - IV. GraphML and GEXF are plain two-column text formats with no support for attributes.
- A. I, II, III
 - B. I, II, IV
 - C. II, III, IV
 - D. I, III

Answer: A (statements I, II, III)

Reference: Lecture 19 – Datasets: Different Formats (≈04:16–17:07)

Solution: I and II describe the edge-list and adjacency-list layouts. III is the key trade-off: the simple list formats sacrifice flexibility and cannot carry labels or attributes, whereas GML, GraphML and GEXF can. IV is false — GraphML and GEXF are XML-based, attribute-rich formats, not plain two-column text.

Q4. [MCQ] The food-web GraphML file declares directed edges (predator → prey); the air-traffic file is a plain undirected edge list. What does the snippet print?

```
fw = nx.read_graphml('food_web.graphml')
at = nx.read_edgelist('air_traffic.txt')
print(nx.is_directed(fw), nx.is_directed(at))
```

- A. True False
- B. False True
- C. True True
- D. False False

Answer: A

Reference: Lecture 21 – Datasets: Analyzing Using NetworkX (≈07:35–09:24)

Solution: `nx.is_directed` returns a Boolean. The food-web GraphML declares directed edges, so it loads as a directed graph and the first value is True; the air-traffic edge list is undirected, so the second value is False. The line printed is True False.

Q5. [MCQ] Using his eight-station sketch (links A–B, A–C, A–D, B–C, D–E, E–F, F–G, G–H, E–G), Tomás builds the degree distribution the way the lecture does. With the distinct degrees sorted on the x-axis, what is the list of counts on the y-axis?

```
all_degrees = list(nx.degree(G).values())
unique_degrees = sorted(set(all_degrees))          # x-axis
counts = [all_degrees.count(d) for d in unique_degrees]  # y-axis
```

- A. [1, 4, 3]
- B. [1, 2, 3]
- C. [3, 4, 1]
- D. [8, 8, 8]

Answer: A

Reference: Lecture 21 – Datasets: Analyzing Using NetworkX (≈15:00–23:20)

Solution: Counting links per station: A, E and G have degree 3; B, C, D and F have degree 2; H has degree 1. The distinct degrees are [1, 2, 3] (the x-axis). Degree 1 occurs once, degree 2 four times, degree 3 three times, so counts = [1, 4, 3] — the number of nodes at each degree, plotted on the y-axis.

Q6. [MCQ] For that same eight-station sketch (8 nodes, 9 links), what does `nx.density` return, and what would density be for an empty graph and for a complete graph?

- A. $9/28 \approx 0.32$; density is 0 for an empty graph and 1 for a complete graph.
- B. $9/8 \approx 1.13$; density is always greater than 1.
- C. $9/64 \approx 0.14$; density is 1 for an empty graph and 0 for a complete graph.
- D. $8/9 \approx 0.89$; density equals the average degree.

Answer: A

Reference: Lecture 21 – Datasets: Analyzing Using NetworkX (≈27:27–30:40)

Solution: Density is the fraction of possible edges that are present: edges / $C(n,2)$. Here that is $9 / [8 \cdot 7 / 2] = 9/28 \approx 0.32$. For a simple graph density lies in [0, 1], equalling 0 when there are no edges and 1 when all $C(n,2)$ edges are present.

Q7. [MCQ] In the sketch, station A is linked to B, C and D. Among those three neighbours, only the link B–C exists. What is A's clustering coefficient?

- A. $1/3 \approx 0.33$
- B. $1/6 \approx 0.17$
- C. $1/2 = 0.50$
- D. $3/8 = 0.375$

Answer: A

Reference: Lecture 21 – Datasets: Analyzing Using NetworkX (≈30:55–34:26)

Solution: A node's clustering coefficient is (links present among its neighbours) / (links possible among them). A has 3 neighbours, so the possible links number $C(3,2) = 3$, and only 1 (B–C) is present, giving $1/3 \approx 0.33$.

Q8. [MCQ] In the sketch, the two most widely separated stations are B and H (and likewise C and H), reachable only by a 5-link shortest path such as B–A–D–E–G–H. What is `nx.diameter` for this network, and what does it represent?

- A. 9 — the number of links.
- B. 5 — the largest of all shortest-path lengths, i.e. the distance between the two most separated stations.
- C. 8 — the number of stations.
- D. 3 — the average number of links per station.

Answer: B

Reference: Lecture 21 – Datasets: Analyzing Using NetworkX (≈34:26–35:54)

Solution: Diameter is the maximum over all node pairs of the shortest-path length — the distance between the two most distant nodes. The B–H (and C–H) shortest path has length 5 and nothing is farther, so the diameter is 5. It is not a link count (A), a node count (C), or an average degree (D).

Q9. [MCQ] Tomás finds that the air-traffic network is connected (`nx.is_connected` returns True). Operationally, what does that guarantee?

- A. Every airport has a non-stop route to every other airport.
- B. From any airport you can reach any other airport using one or more flights, possibly with transfers.
- C. Every airport has the same number of routes.
- D. The network contains no hub airports.

Answer: B

Reference: Lecture 21 – Datasets: Analyzing Using NetworkX (connectivity)

Solution: Connectedness means a path exists between every pair of nodes — so from any airport you can reach any other by a sequence of flights, with transfers if needed (B). A would require a direct route between every pair (a complete graph); connectivity says nothing about equal route counts (C) or the absence of hubs (D).

Q10. [MSQ] Reflecting on what structural analysis of real networks usually reveals, which statements are correct? (Select all that apply.)

- I. A power-law degree distribution means a few nodes have very high degree (hubs) while most nodes have low degree.
 - II. Plotted on log–log axes, a power-law degree distribution appears approximately as a straight line.
 - III. The density of a simple graph lies between 0 (no edges) and 1 (complete graph).
 - IV. Real-world networks typically have very large diameters that grow in proportion to the number of nodes.
- A. I, II, III

- B. I, II, IV
- C. II, III, IV
- D. I, III

Answer: A (statements I, II, III)

Reference: Lecture 21 – Datasets: Analyzing Using NetworkX (≈25:06, density & diameter)

Solution: I and II describe the power-law degree distribution and its straight-line signature on log–log axes. III states the density bounds for a simple graph. IV is false — real networks tend to have surprisingly small diameters, because the many connections keep nodes close together.

Comprehension 2 — How Many Links Until the Sensor Mesh Connects?

Case Study

A field-ecology team scatters 50 solar-powered sensors across a nature reserve. At first every sensor stands alone. Over the following weeks, pairs of sensors that happen to fall within radio range of one another switch on a link, essentially at random, one pair at a time. The team needs to know how many such links must appear before the mesh becomes fully connected — the point at which a reading taken anywhere can hop, link by link, all the way to the base station. Maya studies this with a probability argument and a short simulation.

She first reasons about why the mesh almost never ends up split into two separate clusters with no link between them. For that to happen, every one of the many possible links that could bridge the two clusters would have to be missing at the same time. With so many independent chances to bridge the gap, all of them failing is about as plausible as tossing a fair coin a few hundred times and never once seeing heads — conceivable, but not something that happens in practice. So as links accumulate, a permanent split becomes vanishingly unlikely.

To pin down how many links are actually needed, Maya borrows the lecture’s tactic of tracking the last “lonely” sensor — one that still has no link at all. A single new link involves any one particular sensor only with a small chance, and a sensor stays lonely only if every link formed so far has avoided it. Multiplying those chances together and leaning on a standard calculus limit, she estimates the number of random links at which lonely sensors essentially disappear — and, once none are left, a split-off cluster is itself extremely unlikely, so the mesh is almost surely whole.

Finally she codes a simulation, `connectivity.py`. It starts from a set of unlinked sensors, repeatedly switches on one random link between two distinct sensors until NetworkX reports the mesh is connected, then returns how many links were needed. By averaging this over many runs and many mesh sizes, she means to compare how the link count grows against a candidate formula. The core routines are:

```
import networkx as nx, random

def add_nodes(n):
    G = nx.Graph()
    G.add_nodes_from(range(0, n))
    return G

def add_random_edge(G):
    v1 = random.choice(list(G.nodes()))
    v2 = random.choice(list(G.nodes()))
    if v1 != v2:
        G.add_edge(v1, v2)
    return G

def create_instance(n):
    G = add_nodes(n)
    while nx.is_connected(G) == False:
```

```
G = add_random_edge(G)
return G.number_of_edges()
```

Questions 1–10

Q1. [MCQ] How many distinct sensor-to-sensor links are possible in the 50-sensor mesh — that is, what is $C(50,2)$?

- A. 2500
- B. 1225
- C. 2450
- D. 50

Answer: B

Reference: Lecture 23 – Emergence of Connectedness (≈05:12); Lecture 24 (≈00:21)

Solution: $C(50,2) = 50 \cdot 49 / 2 = 1225$ — each unordered pair of sensors counted once. Option C (2450) is the ordered count $50 \cdot 49$ without halving; A and D are unrelated.

Q2. [MCQ] Suppose the mesh threatened to split into a cluster of 30 sensors and a cluster of 20, with no link across. How many across-cluster links are possible, and how does that compare with the total number of possible links?

- A. 50 — one per sensor.
- B. 600 — close to half of all 1225 possible links.
- C. 1225 — all of the possible links.
- D. 10 — the difference between the cluster sizes.

Answer: B

Reference: Lecture 23 – Emergence of Connectedness (≈05:12)

Solution: Each of the 30 sensors in one cluster could link to each of the 20 in the other, giving $30 \times 20 = 600$ possible across-cluster links — close to half of the 1225 possible links overall. For the split to survive, all 600 must be absent, which is what makes a clean split so improbable.

Q3. [MCQ] Why does Maya conclude that a permanent two-cluster split is vanishingly unlikely once many links have formed?

- A. Because the sensors are deliberately linked so as to keep the mesh whole.
- B. Because hundreds of possible across-cluster links each have an independent chance to appear, and for the split to persist every single one would have to stay missing — about as implausible as flipping a fair coin hundreds of times and never getting heads.
- C. Because a fully linked mesh is the only connected arrangement of 50 sensors.
- D. Because the two clusters have different sizes, which forces a link between them.

Answer: B

Reference: Lecture 23 – Emergence of Connectedness (≈02:23–05:12)

Solution: Roughly half of all possible links run across the two clusters; for a split to persist, random link formation would have to miss every one of them — a long string of independent chances all failing, like

never flipping heads in hundreds of tosses. Links form at random, not by design (A); connectivity does not require a fully linked mesh (C); and unequal cluster sizes force nothing (D).

Q4. [MCQ] In Maya's argument, one new link joins two of the n sensors. What is the probability that this single link avoids a particular sensor v , and the probability that v is still lonely after K independent links?

- A. Avoids with probability $2/n$; lonely after K links with probability $(2/n)^K$.
- B. Avoids with probability $1 - 2/n$; lonely after K links with probability $(1 - 2/n)^K$.
- C. Avoids with probability $1/n$; lonely after K links with probability $(1/n)^K$.
- D. Avoids with probability $1 - 2/n$; lonely after K links with probability $K \times (1 - 2/n)$.

Answer: B

Reference: Lecture 24 – Advanced Material: Emergence of Connectedness (≈11:01–13:06)

Solution: A link picks 2 of the n sensors, so the chance it lands on v is $2/n$ and the chance it avoids v is $1 - 2/n$. Because the K links are independent, the avoid-chances multiply: $P(v \text{ still lonely after } K \text{ links}) = (1 - 2/n)^K$. Probabilities multiply, not add, ruling out D.

Q5. [MSQ] Which statements about the lonely-sensor argument are correct? (Select all that apply.)

- I. $P(\text{a single new link avoids a fixed sensor } v) = 1 - 2/n$.
 - II. After K independent links, $P(v \text{ is still lonely}) = (1 - 2/n)^K$.
 - III. Setting $K = n \cdot \log n$ drives $P(v \text{ lonely})$ to about $1/n^2$, which $\rightarrow 0$ as n grows large.
 - IV. The argument proves the mesh is connected after exactly n links, for every n .
- A. I, II, III
 - B. I, II, IV
 - C. II, III, IV
 - D. I, III

Answer: A (statements I, II, III)

Reference: Lecture 24 – Advanced Material: Emergence of Connectedness (≈11:01–16:21)

Solution: I and II are the avoid-probability and its K -fold product. III is the punchline: with $K = n \cdot \log n$ the loneliness probability collapses to about $1/n^2$, which vanishes as n grows, so no sensor stays lonely. IV is false — the result concerns the order $n \cdot \log n$ (with high probability), not exactly n links with certainty.

Q6. [MCQ] Using the limit $(1 - 1/m)^m \rightarrow 1/e$, set $K = n$ links. The probability that a sensor is still lonely, $(1 - 2/n)^n$, approaches which value (numerically about 0.13 for $n = 50$)?

- A. $1/e \approx 0.37$
- B. $1/e^2 \approx 0.14$
- C. $e^2 \approx 7.39$
- D. 0, exactly

Answer: B

Reference: Lecture 24 – Advanced Material: Emergence of Connectedness (≈13:06–16:21)

Solution: Rewrite $(1 - 2/n)^n = [(1 - 1/(n/2))^{(n/2)}]^2 \rightarrow (1/e)^2 = 1/e^2 \approx 0.135$. Numerically $0.96^{50} \approx 0.13$, matching $1/e^2$. So after only n links a fixed sensor still has a non-trivial chance of being lonely — which is why the argument pushes on to $n \cdot \log n$ links.

Q7. [MCQ] The analysis concludes that of the order of $n \cdot \log n$ random links connect an n -sensor mesh. Why is this described as “almost as good as n ”?

- A. Because $n \cdot \log n$ is roughly n^2 , so the link requirement grows quadratically with the number of sensors.
- B. Because $\log n$ grows very slowly (about the number of digits in n), so $n \cdot \log n$ is only a small multiple of n — a near-linear number of links suffices.
- C. Because $\log n$ grows faster than n , so $n \cdot \log n$ exceeds the total $C(n,2)$ possible links.
- D. Because exactly n links always connect any n -sensor mesh.

Answer: B

Reference: Lecture 24 – Advanced Material: Emergence of Connectedness (closing remarks)

Solution: $\log n$ grows very slowly — the lecture likens it to the number of digits in n — so $n \cdot \log n$ is only a modest factor above n : a near-linear (not quadratic) number of links connects the mesh. A is wrong ($n \cdot \log n \ll n^2$), C is wrong ($\log n$ grows far slower than n), and D overstates the result, which is of the order $n \cdot \log n$, not exactly n .

Q8. [MCQ] Consider the helper function `add_nodes` from `connectivity.py` (shown above). What is true immediately after `G = add_nodes(50)`?

- A. G has 50 sensors and 0 links, and `nx.is_connected(G)` returns False.
- B. G has 50 sensors already linked in a chain, and `nx.is_connected(G)` returns True.
- C. G has 50 sensors and 1225 links (a fully linked mesh).
- D. G is empty: it has 0 sensors and 0 links.

Answer: A

Reference: Lecture 25 – Programming Illustration: Emergence of Connectedness (≈01:26–06:00)

Solution: `add_nodes(50)` creates an empty graph and adds nodes 0–49 with `add_nodes_from(range(0,50))` — no links are added. So G has 50 sensors and 0 links, and a 50-node graph with no links is disconnected, so `nx.is_connected(G)` returns False. This is the starting point before links are switched on one at a time.

Q9. [MCQ] In `add_random_edge` two sensors are drawn with `random.choice` and a link is added only inside an `if v1 != v2` check. Why is the check there, and when does the loop in `create_instance` stop?

- A. The check prevents adding a self-link when the same sensor is drawn twice; the while loop stops the moment `nx.is_connected(G)` becomes True.
- B. The check prevents adding a duplicate link; the while loop stops after exactly n iterations.
- C. The check ensures the two sensors are in different clusters; the loop stops when the mesh is fully linked.
- D. The check is unnecessary and the loop never terminates.

Answer: A

Reference: Lecture 25 – Programming Illustration: Emergence of Connectedness (≈06:00–11:00)

Solution: `random.choice` can return the same sensor for `v1` and `v2`; the `if v1 != v2` guard skips that case so no self-link is created. `create_instance` keeps calling `add_random_edge` while `nx.is_connected(G) == False`, so it halts the instant the mesh first becomes connected — not after a fixed count and not at full linkage.

Q10. [MSQ] Which statements about the simulation and its conclusion are correct? (Select all that apply.)

- I. `create_instance(n)` returns the number of links present at the moment the mesh first becomes connected.
- II. Averaging the link count over many runs smooths the random spikes in the curve.
- III. The measured link-count curve closely follows $(n \cdot \log n)/2$, consistent with “of the order $n \cdot \log n$.”
- IV. The experiment shows that about n^2 links are needed to connect an n -sensor mesh.

- A. I, II, III
- B. I, II, IV
- C. II, III, IV
- D. I, III

Answer: A (statements I, II, III)

Reference: Lecture 25 – Programming Illustration: Emergence of Connectedness (≈11:00–29:46)

Solution: I — the function returns `G.number_of_edges()` exactly when the while loop exits at connectivity. II — a single random run is spiky, so averaging over many runs smooths the curve. III — the screencast finds the curve hugs $(n \cdot \log n)/2$, i.e. of order $n \cdot \log n$. IV is false — the whole point is that a near-linear $n \cdot \log n$ number of links suffices, far fewer than n^2 .